
Network Analysis, Information Spread & Influence Maximisation

A Report Submitted to
SVKM's
Narsee Monjee Institute of Management Studies (NMIMS)

**As a part of Semester III
Of
Bachelor of Science Applied Mathematical Computing
By**

Tanushri Shetty (A011)

Certificate

This is to certify that work described in this thesis entitled “.....” has been carried out by Tanushri Shetty under my supervision. I certify that this is his/her bonafide work. The work described is original and has not been submitted for any degree to this or any other University.

Date:

Place:

(Supervisor)

Statement of Certificate

This is to submit that this written submission in my report entitled “*Network Analysis, Information Spread and Influence Maximisation*” (Semester III) represents my ideas in my own words and where others’ ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I stuck to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/ data/ fact/ source in my submission. I understand that any violation of the above will be cause for disciplinary action by the School and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Contents

Abstract	5
Introduction	6
Network Theory	7
• Basic Components of a Network	7
• Types of Graphs	8
• Real World Networks	9
• Average Path Length	10
• Centrality Measures	11
Building a Network	14
Louvain Algorithm for Community Detection	18
Information Flow	21
Influence Maximisation	26
Conclusion	28
Bibliography	29

Abstract

In this project, we delve into the magical universe of J.K. Rowling's first Harry Potter book to explore the intricate relationships between characters using network analysis techniques. We construct a dynamic network where characters are nodes, and connections are formed when two characters co-occur within a sliding window of 5 lines throughout the entire text.

Applying the Louvain algorithm for community detection, we uncover clusters of characters that share significant narrative proximity. This approach allows us to reveal latent relationships and affiliations among characters, providing insights into the social fabric of the wizarding world. To model the spread of information and capture the dynamics of rumour propagation, we employ the independent cascade model. This simulation enables us to study how information travels through the character network, highlighting influential characters and the pathways of information diffusion.

Taking a step further, we address the influence maximisation problem within our constructed network. By leveraging graph algorithms, we identify a pair of characters whose joint influence is maximised in spreading information throughout the network. This investigation sheds light on pivotal characters whose actions and connections play a crucial role in shaping the narrative landscape of the Harry Potter universe.

In summary, our project offers a comprehensive analysis of character relationships, information spread dynamics, and influence maximisation within the context of the first Harry Potter book. This project provides us valuable insights about social networks and also demonstrates the applicability of network analysis techniques in uncovering hidden structures and dynamics within complex systems.

Introduction

In the intricate landscape of interconnected systems, understanding the dynamics of relationships, information flow, and influence is paramount. This project embarks on a journey into the realm of network analysis, information spread, and influence maximisation, seeking to unravel the intricate threads that weave through complex networks.

Networks serve as powerful representations of relationships, be it social connections, communication channels, or collaborative interactions. Through advanced network analysis techniques, this project aims to dissect and comprehend the underlying structures within these intricate webs.

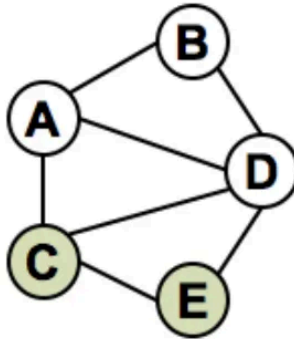
This project aims to provide accessible introductions to key concepts and ideas regarding network analysis, information spread and influence maximisation such as community detection, rumour spread dynamics using the independent cascade model of information flow and the influence maximisation problem used frequently in marketing.

Network Theory

Basic Components of a Network



Nodes



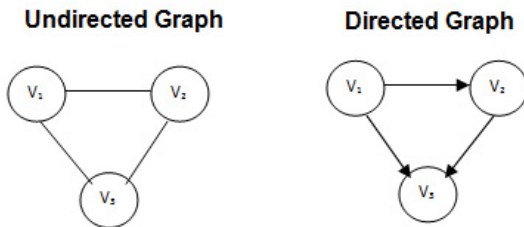
Edges

Nodes (A,B,C,D,E in the example) are usually representing entities in the network, and can hold self-properties (such as weight, size, position and any other attribute) and network-based properties (such as *Degree*- number of neighbours or *Cluster*- a connected component the node belongs to etc.).

Edges represent the connections between the nodes, and might hold properties as well (such as weight representing the strength of the connection, direction in case of asymmetric relation or time if applicable).

These two basic elements can describe multiple phenomena, such as *social connections*, *virtual routing network*, *physical electricity networks*, *roads network*, *biology relations network* and many other relationships.

Types of Graphs



1. Undirected Graph:

In an undirected graph:

- Each edge is an unordered pair of nodes.
- If there is an edge between nodes A and B, it implies that the relationship is mutual, and you can travel from A to B or from B to A without any distinction.
- The edge connecting nodes A and B does not have an inherent direction.
- Represented by a set of vertices and a set of edges, where each edge is an unordered pair $\{A, B\}$.
- Undirected Social Network:
 - In an undirected social network, edges between individuals represent mutual relationships. If person A is connected to person B, it implies that the relationship is reciprocal, and person B is also connected to person A.
 - This representation is suitable when the relationships in the social network are symmetric, such as friendships. If A is friends with B, then B is automatically friends with A.

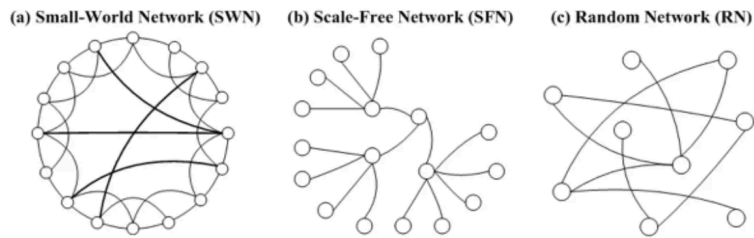
2. Directed Graph:

In a directed graph

- Each edge is an ordered pair of vertices (nodes), indicating a one-way relationship or direction.
- If there is a directed edge from A to B, it does not necessarily imply a directed edge from B to A.
- The edge connecting nodes A and B has a specific direction: $A \rightarrow B$.
- Represented by a set of vertices and a set of directed edges, where each directed edge is an ordered pair (A, B) .
- Directed Social Network:
 - In a directed social network, edges have a specific direction, indicating the direction of influence or communication. If person A is connected to person B, it does not necessarily imply that B is connected to A.
 - This representation is appropriate when the relationships have a clear direction, such as following relationships on social media platforms. If A follows B, it doesn't mean that B automatically follows A.

Real World Networks

Real-world networks and in particular social networks have a unique structure which often differs them from random mathematical networks:



- **Small World** phenomenon claims that real networks often have very short paths (in terms of number of hops) between any connected network members. This applies for real and virtual social networks (the six handshakes theory) and for physical networks such as airports or electricity or web-traffic routings.
- **Scale Free networks** with power-law degree distribution have a skewed population with a few highly-connected nodes (such as social-influences) and a lot of loosely-connected nodes.
- **Random networks** are networks where connections between nodes are established randomly.

Average Path Length

The average path length is a metric used in network analysis to quantify how well-connected or reachable a network is. It measures the average number of steps it takes to traverse from one node to another, considering all possible pairs of nodes in the network. A lower average path length generally indicates a more efficient and well-connected network.

Here's how you can calculate the average path length of a network:

Shortest Paths:

- First, find the shortest paths between all pairs of nodes in the network. The shortest path is the minimum number of edges or steps needed to travel from one node to another.

Path Length:

- Calculate the path length for each pair of nodes, i.e., the number of edges in the shortest path.

Average Path Length:

- Compute the average of all the path lengths.

Formula:

The average path length (L) is given by the formula:

$$L = \frac{1}{N(N-1)} \sum_{i \neq j} d_{ij}$$

where:

- N is the number of nodes in the network.
- d_{ij} is the shortest path length from node i to node j

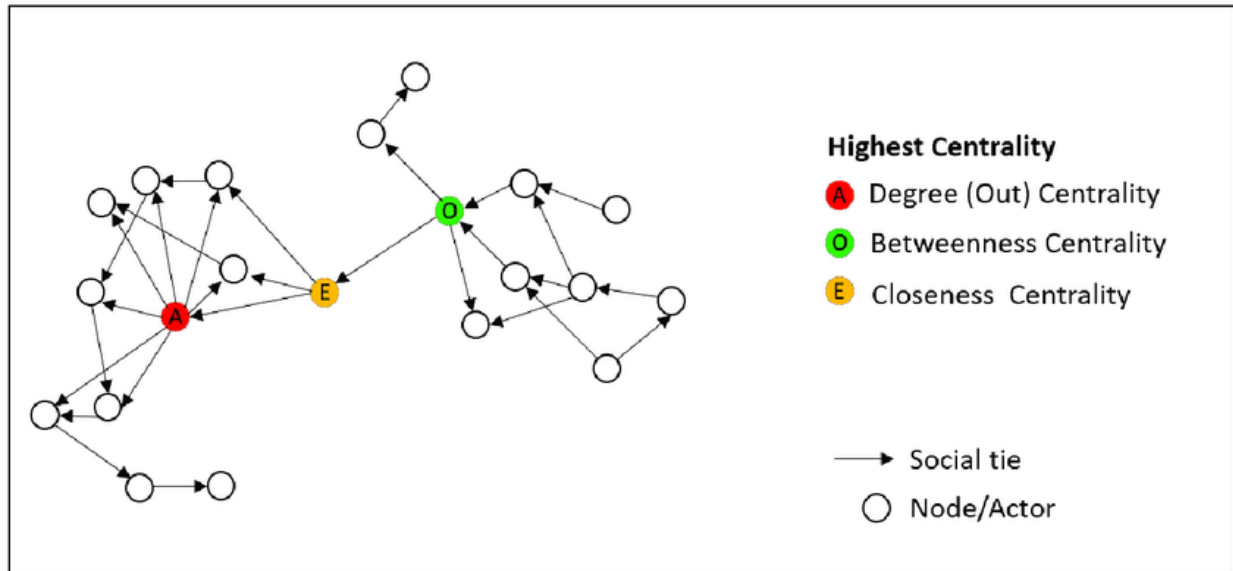
Key Points:

- **Efficiency Measure:** A lower average path length signifies a more efficient and well-connected network. It implies that nodes are generally reachable with fewer steps.
- **Scale-Free Networks:** Some real-world networks, like social networks and the World Wide Web, exhibit a small-world property. They have a low average path length despite having a large number of nodes.
- **Random Networks:** In contrast, random networks tend to have longer average path lengths. Small-world networks strike a balance between regular (ordered) and random (disordered) networks.
- **Network Resilience:** The average path length is related to a network's resilience. Networks with a small average path length are often more robust against random failures.

Centrality Measures

Highly central nodes play a key role in a network, serving as hubs for different network dynamics. However the definition and importance of centrality might differ from case to case, and may refer to different centrality measures:

1. Degree
2. Closeness
3. Betweenness



1. Degree Centrality

Degree centrality is a measure used in graph theory to quantify the importance or centrality of a node within a network. It is based on the number of edges a node has, which is referred to as its degree. In a directed graph, nodes have both an in-degree (number of incoming edges) and an out-degree (number of outgoing edges).

The degree centrality of a node v in an undirected graph is calculated as:

$$\text{Degree Centrality}(v) = \frac{\text{Degree of } v}{\text{Total number of nodes} - 1}$$

In a directed graph, you may calculate in-degree centrality, out-degree centrality, or a combination of both, depending on your specific interest.

- In-degree Centrality: Measures how many edges are directed towards a node.

$$\text{In-degree Centrality}(v) = \frac{\text{In-degree of } v}{\text{Total number of nodes} - 1}$$

-
- Out-degree Centrality: Measures how many edges are directed away from a node.

$$\text{Out-degree Centrality}(v) = \frac{\text{Out-degree of } v}{\text{Total number of nodes} - 1}$$

Key Points:

- High Degree Centrality:
 - Nodes with high degree centrality are often considered more central or important in the network. In social networks, high degree centrality may indicate popularity or influence.
- Connection to Small World Phenomenon:
 - In networks exhibiting the small-world phenomenon, most nodes have relatively high degree centrality, contributing to short average path lengths between nodes.
- Application in Identifying Hubs:
 - Degree centrality is instrumental in identifying hub nodes—those with exceptionally high degrees. Hubs play a crucial role in network connectivity.
- Limitations:
 - Degree centrality might not fully capture a node's importance, especially in networks where the strength of connections matters. For instance, a node might have a few strong connections, or weighted connections, making it influential even with a lower degree.

2. Closeness

Closeness centrality is a measure in graph theory that quantifies how central or close a node is to other nodes in a network. It is based on the concept of the average shortest path length, representing how quickly information or influence can spread from a particular node to all other nodes in the network.

The closeness centrality of a node v is calculated as the reciprocal of the average shortest path length from that node to all other nodes in the network.

$$\text{Closeness Centrality}(v) = \frac{1}{\text{Average Shortest Path Length from } v}$$

Key Points:

- Interpretation:
 - High closeness centrality indicates that a node is, on average, close to all other nodes in the network in terms of the shortest path length. It implies efficient access to other nodes.

-
- Calculation of Average Shortest Path Length:
 - The average shortest path length from a node v to all other nodes is calculated by finding the shortest paths from v to all other nodes and then taking the average.
 - Application in Real-world Networks:
 - Closeness centrality is often used in the analysis of transportation networks, social networks, and communication networks, where efficient information flow is crucial.

3. Betweenness

Betweenness centrality is a measure in graph theory that quantifies the extent to which a node lies on the shortest paths between other nodes in a network. It is a way of identifying nodes that act as intermediaries or bridges in the flow of information or influence between different parts of the network.

The betweenness centrality of a node v is calculated based on the number of shortest paths that pass through that node. The formula is as follows:

$$\text{Betweenness Centrality}(v) = \sum_{s \neq v \neq t} \frac{\text{Number of shortest paths from } s \text{ to } t \text{ passing through } v}{\text{Number of shortest paths from } s \text{ to } t}$$

Key Points:

- Interpretation:
 - High betweenness centrality indicates that a node is frequently traversed by shortest paths between other nodes. Such nodes play a critical role in maintaining the network's connectivity.
- Application in Network Resilience:
 - Nodes with high betweenness centrality are often crucial for maintaining the resilience of a network. Their removal might disrupt the flow of information between different parts of the network.
- Identification of Key Nodes:
 - Nodes with high betweenness centrality are often considered key connectors or bottlenecks in the network. They can have significant influence or control over the information flow.

Building a Network



Networks can be constructed from various datasets, as long as we're able to describe the relations between nodes. In this project we'll build and visualise the network of characters of the last Harry Potter book Harry Potter and the Deathly Hallows with the Python *networkx* package.

Using the python library *BeautifulSoup* we'll scrape the web for a list of characters of the book and convert that into a *pandas* dataframe.

```
#Finding characters using html tag
table = soup.find_all("h3")

#Adding characters to a list
data = []

for i in table:
    character = i.text
    data.append({"character": character})

#Converting list to dataframe
character_df = pd.DataFrame(data)
```

Using the natural language processing python library *spacy* we'll visualise the text and then create a dataframe containing all the sentences and the entities of each sentence.

```
#Opening book using spacy
book = all_books[0]
book_text = open(book,encoding="ISO-8859-1").read()
book_doc = NER(book_text)

#Creating dataframe for sentence and entities
sent_entity_df = []

for sent in book_doc.sents:
    entity_list = [ent.text for ent in sent.ents]
    sent_entity_df.append({"sentence": sent, "entities": entity_list})

sent_entity_df = pd.DataFrame(sent_entity_df)
sent_entity_df.head(5)
```

	sentence	entities
0	(Chapter 1, \n, The, Dark, Lord, Ascending, \...	[Chapter 1, two, moonlit lane]
1	(For, a, second, they, stood, quite, still, ,,...	[second]
2	(â€ News?â€ , asked, the, taller, of, the, two...	[two, Severus Snape]
3	(The, lane, was, bordered, on, the, left, by, ...	[]
4	(The, menâ€ s, long, cloaks, flapped, around, ...	[]

We'll then filter out the characters from the entities.

```
#Function to return full name even if last name or first name appears
def match_entities(entities):
    matched_names = []
    for entity in entities:
        for index, row in character_df.iterrows():
            if not pd.isna(row['first_name']) and row['first_name'] in entity:
                matched_names.append(row['full_name'])
            elif not pd.isna(row['last_name']) and row['last_name'] in entity:
                matched_names.append(row['full_name'])
    return matched_names

# Apply the function to create the new column
sent_entity_df_filtered['character_entities'] = sent_entity_df['entities'].apply(match_entities)

# Display the resulting dataframe
sent_entity_df_filtered
```

	sentence	entities	character_entities
2	(â€ News?â€ , asked, the, taller, of, the, two...	[two, Severus Snape]	[Severus Snape]
5	(â€ Thought, I, might, be, late, ,, â€ , said,...	[Yaxley]	[]
8	(You, sound, confident, that, your, reception,...	[good?â€ , Snape]	[Severus Snape]
13	(There, was, a, rustle, somewhere, to, their, ...	[Yaxley]	[]
14	(â€ He, always, did, himself, well, ,, Lucius, .)	[â€ He, Lucius]	[Lucius Malfoy]

Then we'll run a 5 sentence window throughout the text and any two characters appearing in the window would be considered to have a connection and the number of occurrences would be the value or weight of the connection.

```
#Create running window
window_size = 5
relationships = []

for i in range(sent_entity_df_filtered.index[-1]):
    end_i = min(i+5, sent_entity_df_filtered.index[-1])
    char_list = sum((sent_entity_df_filtered.loc[i:end_i].character_entities), [])

    char_unique = [char_list[i] for i in range(len(char_list)) if (i==0) or char_list[i] != char_list[i-1]]

    if len(char_unique) > 1:
        for idx, a in enumerate(char_unique[:-1]):
            b = char_unique[idx + 1]
            relationships.append({"source":a,"target":b})

#Create dataframe of relationships and sort it
relationship_df = pd.DataFrame(relationships)
relationship_df = pd.DataFrame(np.sort(relationship_df.values, axis = 1), columns = relationship_df.columns)

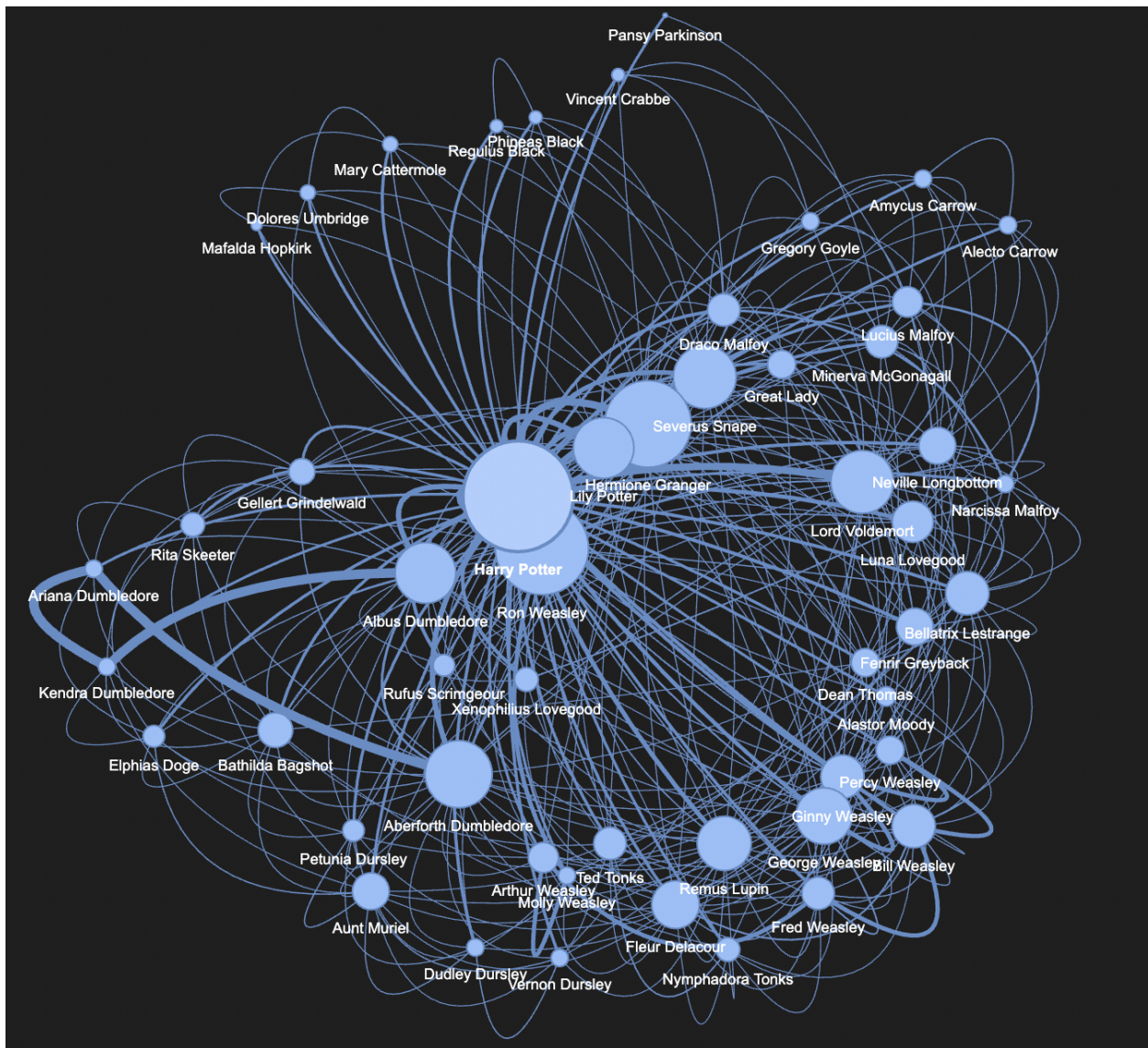
#Create dataframe of relationships with weights of connections
relationship_df["value"] = 1
relationship_df = relationship_df.groupby(["source","target"], sort=False, as_index=False).sum()
relationship_df.head(5)
```

	source	target	value
0	Lucius Malfoy	Severus Snape	9
1	Lord Voldemort	Severus Snape	293
2	Harry Potter	Lord Voldemort	1456
3	Harry Potter	Lily Potter	920
4	Lily Potter	Severus Snape	217

Finally we'll create a graph to visualise the connections using python libraries *Networkx* and *pyvis*.

```
#Creating and visualising an interactive network
from pyvis.network import Network

G = nx.from_pandas_edgelist(relationship_df, source = 'source', target = 'target', edge_attr = 'value',
                           create_using = nx.Graph())
net = Network(notebook = True, width = "1000px", height = "700px", bgcolor = "#222222", font_color = "white")
node_degree = dict(G.degree)
nx.set_node_attributes(G, node_degree, 'size')
net.from_nx(G)
net.repulsion()
net.show("Harry_Potter7.html")
```



Import attributes about the network:

- It is an undirected social network.
- The average path distance of the network is 1.718 which is pretty short. Networks with a small average shortest path length typically exhibit the "small-world" property. This means that, despite having a large number of nodes, the average distance between any two nodes is relatively short. Small-world networks are characterised by a balance between local clustering and global connectivity.
- The highest centrality in the network for all three (i.e Degree, Closeness, Betweenness) is Harry Potter. This makes sense as even if his character is not followed throughout the book, he is referenced very often.

Louvain Algorithm for Community Detection

Louvain is an unsupervised algorithm. It does not require the input of the number of communities nor their sizes before execution. To understand the algorithm we must first understand the concept of modularity.

Modularity

- Modularity measures the strength of a community partition by taking into account the degree distribution.
- Given a network with m edges the given number of edges between two nodes i and j with degrees k_i and k_j respectively is $k_i k_j / 2m$. The strength of a community is given by

$$\sum_{i \in C, j \in C} A_{i,j} - \frac{k_i k_j}{2m}$$

- For a network with S communities and m edges modularity Q is given by

$$Q = \sum_{l=1}^S \sum_{i \in C_l, j \in C_l} A_{i,j} - \frac{k_i k_j}{2m}$$

- A larger value of Q indicates a good community structure.

Steps of the Louvain Algorithm

- Initially, assign each node a unique community such that total nodes = total unique communities
- Now, in an iterative manner, we will try assigning every node 'i' to its neighbouring node 'j' community & recalculate the Modularity of the graph. If modularity improves as compared to when node 'i' wasn't in the 'j' node's community, we will assign 'i' to the community the same as 'j' else not. Remember, j is i's neighbour
- This will be repeated for multiple iterations until unless no further gain is observed in Modularity by moving any node to its neighbour's community & we have reached a Maxima for modularity
- Once done, what we do is create a new graph such that
 1. All nodes in their respective communities are clubbed to form a single node

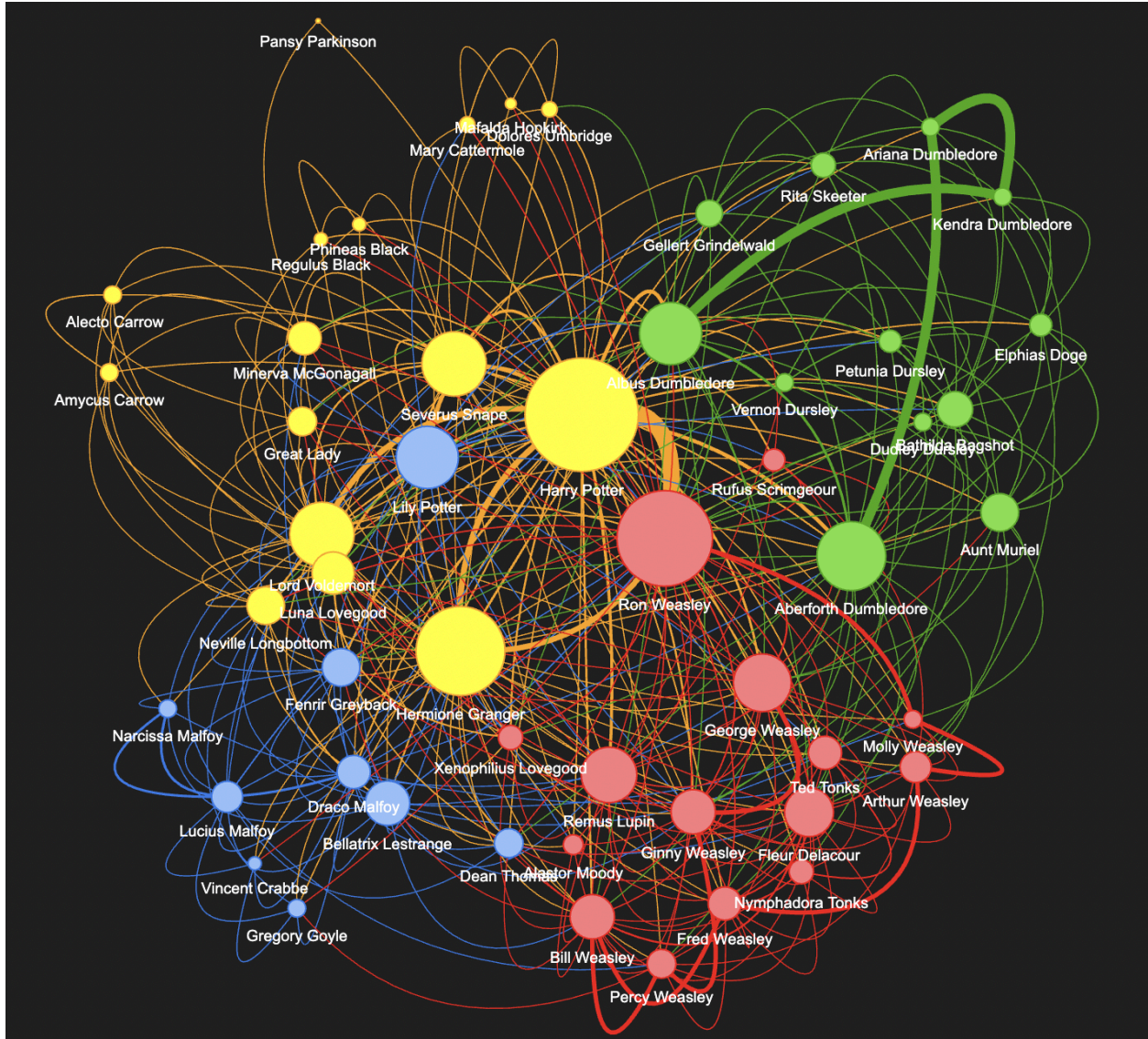
-
2. All edges connecting nodes of community 1 to nodes of community 2 in the previous iteration are clubbed to form a single edge between Community 1 & Community 2 & the edge weight for this new edge is equal:
 - Summation of weights of all edges between nodes of community 1 to nodes of community 2 in case of a directed graph
 - Edge count in case of an unweighted graph
 3. Self edges (like $i - i$) become self edges for their respective communities with weight summation of all such self edges for the nodes of some community '1' as the weight of self-edge for the particular community or count of all self edges in case of an unweighted graph.

We, again go back to the previous iteration where we calculate Modularity for each node of this new graph by moving nodes to the same community as the neighbouring node's community. If Modularity increases, this node is assigned to the neighbour's community as earlier else not & this process continues until no further change in communities is observed.

Running the Louvain algorithm for community detection on our network we get the following results

```
import community as community_louvain

# Run the Louvain algorithm
partition = community_louvain.best_partition(G)
nx.set_node_attributes(G, partition, 'group')
com_net = Network(notebook = True, width = "1000px", height = "700px", bgcolor = "#222222", font_color = "white")
com_net.from_nx(G)
com_net.repulsion()
com_net.show("harry_potter_comm1.html")
```

Here we can observe that characters that have something in common tend to appear within the same community.

- Characters part of the Weasley family and Dumbledore's army tend to appear in the red community.
- Characters who are death eaters and confidants of Voldemort tend to appear in the blue community.
- Characters part of the Dumbledore family and related to them tend to appear in the green community.
- A lot of the central characters like Harry Potter, Hermione Granger, Lord Voldemort and Severus Snape tend to appear in the yellow community.

Information Flow

Information diffusion process may resemble a viral spread of a disease, following contagious dynamics of hopping from one individual to his social neighbours. In this paper we'll be focusing on the Independent Cascade Model of information flow.

The Independent Cascade Model is a widely used and studied model in the field of information diffusion within networks. It is particularly applicable in contexts such as social networks, where information, influence, or contagion spreads from one node (individual) to another.

Here's a basic explanation of the Independent Cascade Model:

Network Representation:

- The process begins with a network or graph representation, where nodes represent entities (such as individuals in a social network), and edges represent potential connections or relationships between them.

Activation Probability:

- Each edge in the network is associated with a probability, denoting the likelihood that the source node (the one with the information) will activate the target node (a neighbour) and influence it to adopt the information.

Propagation Process:

- The information diffusion process unfolds iteratively. Initially, a set of nodes is "seeded" or activated with the information. These nodes are considered the initial spreaders of the information.
- At each iteration, a node that has already been activated attempts to activate its neighbours with a probability defined by the associated edge probability. This happens independently for each neighbour.
- If a neighbour is successfully activated, it becomes part of the process and can, in turn, activate its neighbours in subsequent iterations.

Termination Condition:

- The process continues until no more activations occur, or until a predefined stopping condition is met.

Outcome Measurement:

- The final outcome of the Independent Cascade Model is typically measured by the set of nodes that have been activated by the end of the diffusion process. This set is often referred to as the "influence" or "cascade" resulting from the initial seed nodes.

Probabilistic Nature:

- One key characteristic of the Independent Cascade Model is its probabilistic nature. The activation of a node is not guaranteed; instead, it occurs with a certain probability, reflecting the uncertainty and stochastic nature of real-world information diffusion.

Information Flow Example

To illustrate this we will use a network similar to the one previously constructed but the following one is based on the first Harry Potter book Harry Potter and the Philosopher's stone to better illustrate the process as the book contains fewer characters.

Relying on the independent cascade model, we'll try to track down rumour spreading dynamics.

Suppose *Ron* knows nothing at the beginning of the process, while his friend, *Neville* and Professor *McGonagall*, know a very important secret about his life. Let's watch how the rumour spreads under the *Independent Cascade* model:

```
infection_times = {}

def independent_cascade(G,t,infection_times):
    #doing a t->t+1 step of independent_cascade simulation
    #each infectious node infects neighbors with probability proportional to the weight
    max_weight = max([e[2]['value'] for e in G.edges(data=True)])
    current_infectious = [n for n in infection_times if infection_times[n]==t]
    for n in current_infectious:
        for v in G.neighbors(n):
            if v not in infection_times:
                if G.get_edge_data(n,v)['value'] >= np.random.random()*max_weight:
                    infection_times[v] = t+1
    return infection_times

def plot_G(G,pos,infection_times,t):
    current_infectious = [n for n in infection_times if infection_times[n]==t]
    plt.figure(figsize = (17,10))
    plt.axis('off')
    plt.title('Harry Potter Network, t={}'.format(t),fontsize = 24)

    for node in G.nodes():
        size = 100*weighted_degrees[node]**0.5
        if node in current_infectious:
            ns = nx.draw_networkx_nodes(G,pos,nodelist=[node], node_size=size, node_color='#feba02')
        elif infection_times.get(node,9999999)<t:
            ns = nx.draw_networkx_nodes(G,pos,nodelist=[node], node_size=size, node_color='#f2f6fa')
        else:
            ns = nx.draw_networkx_nodes(G,pos,nodelist=[node], node_size=size, node_color='#009fe3')
        ns.set_edgcolor('#f2f6fa')
    nx.draw_networkx_labels(G,pos,{n:n.replace('-','\n') for n in G.nodes()},font_size=10);

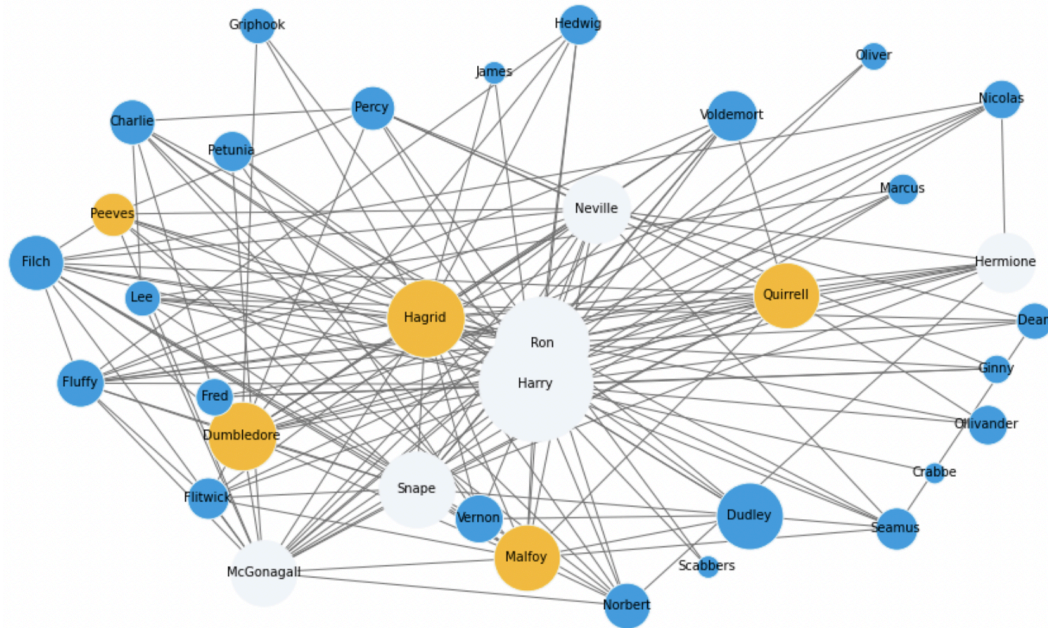
    for e in G.edges(data=True):
        nx.draw_networkx_edges(G,pos,[e],edge_color='#707070')

infection_times = {'Neville':-1,'McGonagall':-1,'Ron':0}

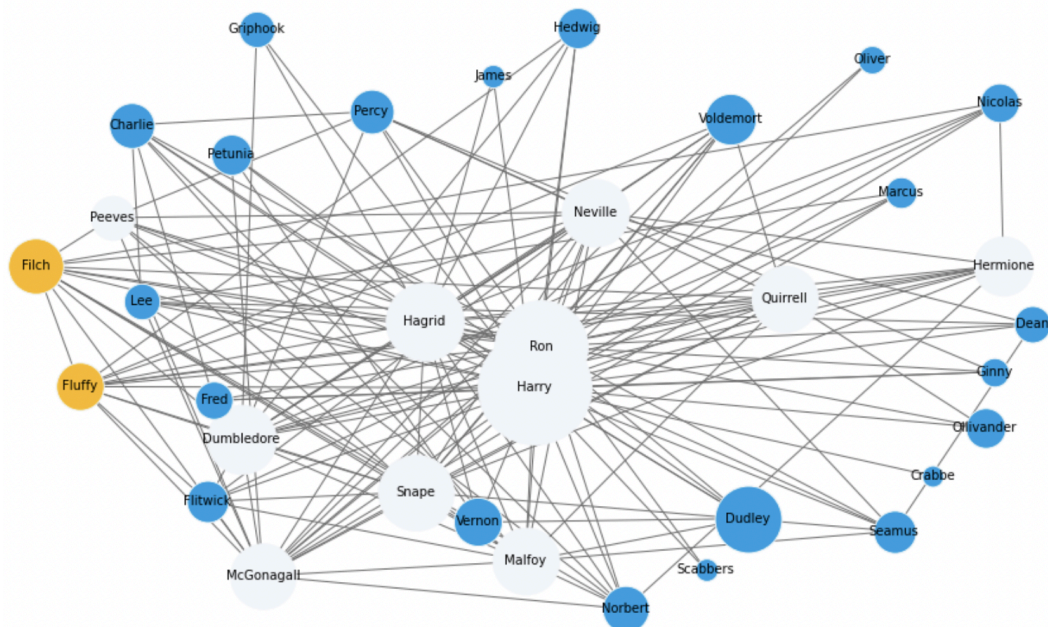
for t in range(5):
    plot_G(G,pos,infection_times,t)
    infection_times = independent_cascade(G,t,infection_times)
```

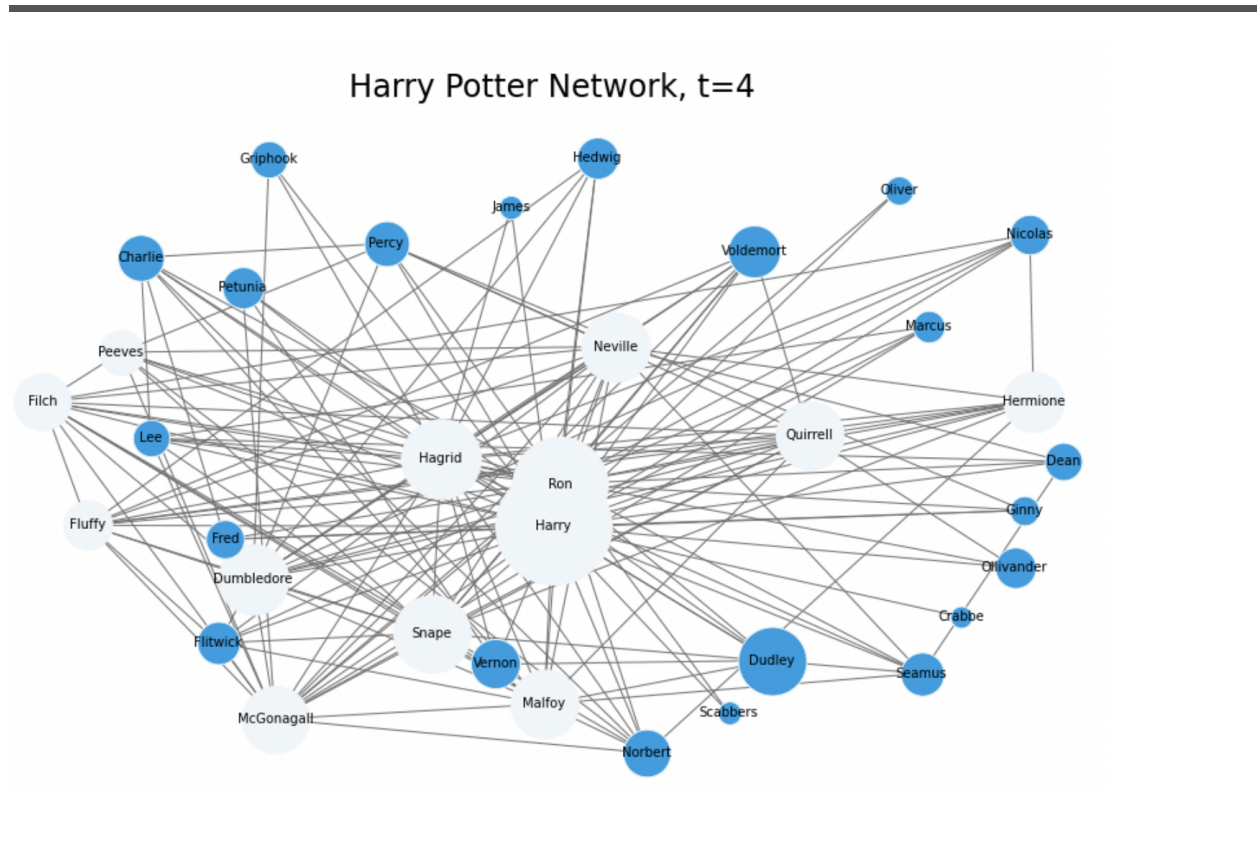
Harry Potter Network, t=0

Harry Potter Network, $t=2$



Harry Potter Network, $t=3$





The rumour reaches *Ron* at $t=1$, spreads to his neighbours in the following time-steps and quickly spreads all around the network, resulting in being public knowledge. Such dynamics are highly dependent on model parameters, which can drive the diffusion process to different patterns.

Influence Maximisation

The influence maximisation problem describes a marketing (but not only) setup, where the goal of the marketer is to select a limited set of nodes in the network (seeding set) such that will naturally spread the influence to as many nodes as possible.

Such influencers can be identified with numerous techniques, the following is the technique used in this project.

1. Network Representation:
 - The code assumes you have a network represented as a graph using the NetworkX library. Nodes in the graph represent entities (characters here), and edges represent relationships between them.
2. Independent Cascade Model:
 - As previously discussed the Independent Cascade Model is a probabilistic model of information diffusion in networks. It assumes that each edge in the network has an associated probability of influence transmission. Here we assume each edge has an equal probability.
3. Simulation Process:
 - The **independent_cascade_simulation** function simulates the spread of influence in the network based on the Independent Cascade Model.
 - It starts with a pair of randomly selected seed nodes. These nodes are the initial sources of influence.
 - The simulation proceeds in iterations. At each iteration, active nodes attempt to activate their neighbours with a certain probability.
 - The process continues until no more activations occur, and the set of active nodes represents the final influence or cascade.
4. Finding Maximum Influence:
 - The **find_max_influence_nodes** function aims to find the pair of seed nodes that results in the maximum influence over multiple simulations.
 - It runs the simulation with different random pairs of seed nodes for a specified number of simulations (num_simulations).
 - The "Maximum Influence" value represents the highest number of nodes influenced in any single simulation.
5. Monte Carlo Simulation:
 - The approach used in the code is a form of Monte Carlo simulation. By running the simulation multiple times with different random seed nodes, the code provides a statistical estimate of the pair of nodes that, on average, leads to the maximum influence.

```

import networkx as nx
import random

def independent_cascade_simulation(graph, seed_nodes):
    active_nodes = set(seed_nodes)
    new_nodes = set(seed_nodes)

    while new_nodes:
        current_nodes = list(new_nodes)
        new_nodes = set()

        for node in current_nodes:
            neighbors = set(graph.neighbors(node)) - active_nodes
            new_nodes.update(neighbors)

        active_nodes.update(new_nodes)

    return len(active_nodes)

def find_max_influence_nodes(graph, num_simulations=1000):
    max_influence = 0
    max_influence_nodes = None

    for _ in range(num_simulations):
        seed_nodes = random.sample(graph.nodes(), 2)
        current_influence = independent_cascade_simulation(graph, seed_nodes)

        if current_influence > max_influence:
            max_influence = current_influence
            max_influence_nodes = seed_nodes

    return max_influence_nodes, max_influence

result_nodes, result_influence = find_max_influence_nodes(G)

print(f"Nodes with Maximum Influence: {result_nodes}")
print(f"Maximum Influence: {result_influence}")

Nodes with Maximum Influence: ['McGonagall', 'Neville']
Maximum Influence: 35

```

Although this approach provides us with a seeding set that can influence all 35 nodes, it is not the most efficient approach. Since it uses the Monte Carlo simulation each time the program is run it provides us with a different seeding set, the result of which can be sometimes hard to comprehend. Influence Maximisation is hard, in fact it is considered an NP hard problem. This leaves us with a lot of room for further development to find algorithms that can solve this in more efficient ways.

Conclusion

In conclusion, this project embarked on a comprehensive exploration of network analysis, information spread, and influence maximisation within the context of the first Harry Potter book.

This endeavour not only illuminated the intricate web of character relationships but also demonstrated the applicability of network analysis techniques in uncovering patterns, dynamics, and influential entities within complex literary narratives. The insights gained from this project not only contribute to the understanding of the fictional world of Harry Potter but also showcase the versatility and relevance of network analysis methodologies in broader domains, ranging from social networks to information dissemination strategies.

The analysis of networks is a powerful and intricate tool with widespread applications across diverse domains, especially within the rapidly expanding realm of social networks. This analytical approach finds relevance in tasks such as optimising marketing influence, detecting fraudulent activities, and enhancing recommender systems. While there exists a multitude of tools and techniques applicable to network datasets, the selection process should be deliberate, considering the distinctive characteristics of both the problem at hand and the specific properties inherent in the network being analysed.

Bibliography

- *Goldenberg, Dmitri. “Social network analysis: From graph theory to applications with python.” PyCon 2019 — 3rd Israeli National Python Conference, Israel, 2019. arXiv preprint arXiv:2102.10014 (2021).*
- *Maximising the spread of influence through a social network — E. Tardos et al.*
- *Efficient influence maximisation in social networks — W. Chen et al.*
- *Independent Cascade and Linear Threshold Models - P. Shakarian et al.*